

SECURITY_FINDINGS_REPORT

Enterprise DevSecOps Security Lab

Author: Jagriti Banerjee

Security Assessment	DevSecOps	Kubernetes	Supply Chain	Runtime Detection
---------------------	-----------	------------	--------------	-------------------

Document Objective	This report documents validated security findings, attack paths, business impact, remediation actions, and retest evidence from the Enterprise DevSecOps private lab. It is structured in a professional security assessment format suitable for portfolio, interview, and recruiter review.
--------------------	--

Identify	Validate	Risk Rate	Remediate	Retest	Report
----------	----------	-----------	-----------	--------	--------

Report Field	Details
Project	Enterprise DevSecOps Security Lab
Assessment Owner	Jagriti Banerjee
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, ArgoCD, GHCR
Assessment Style	Application security, container security, Kubernetes security, GitOps, supply chain, runtime detection
Testing Boundary	Private lab only. No public, third-party, or production systems were targeted.

1. Executive Summary

This Security Findings Report documents the issues intentionally introduced, discovered, validated, and remediated inside the Enterprise DevSecOps Security Lab. The environment simulates a realistic cloud-native delivery pipeline that begins with a vulnerable Python Flask application and expands into Docker, CI/CD security automation, Kubernetes hardening, GitOps deployment, supply-chain controls, runtime threat detection, and monitoring.

The report focuses on technical depth and professional security reasoning. Each finding includes an affected component, attack scenario, CVSS-style risk rating, impact analysis, remediation guidance, and validation evidence. The goal is to show not only that tools were run, but that findings were interpreted, prioritized, fixed, and retested.

Overall Assessment Outcome	The lab progressed from intentionally vulnerable application and Kubernetes states to a layered DevSecOps security model with CI/CD scanning, container hardening, Kubernetes least privilege, NetworkPolicy segmentation, ArgoCD drift control, Cosign signing, Syft SBOMs, OPA Gatekeeper policy enforcement, Falco runtime detection, and Grafana observability.
-----------------------------------	---

Severity	Count	Main Themes
Critical	4	Command execution risk, insecure Kubernetes security context, privileged pod escape pattern, cluster-admin RBAC abuse
High	5	SQL injection, vulnerable dependencies, container hardening gap, network segmentation gap, unsigned image/SBOM gap
Medium	3	GitOps drift, admission-control metadata gap, runtime visibility gap
Low	0	No low-severity findings were prioritized for this formal report

The highest-priority risks were those that could enable command execution, container escape patterns, cluster-wide privilege escalation, or broad unauthorized access. These issues were remediated using secure coding patterns, image hardening, Pod Security restricted controls, least-privilege RBAC, NetworkPolicy enforcement, and policy-as-code.

2. Scope, Rules of Engagement, and Rating Method

This was a controlled private lab assessment. The vulnerable application and attack simulations were created specifically for learning and portfolio demonstration. No external systems were scanned or exploited.

In Scope	Out of Scope
Flask routes, dependencies, Dockerfile, image scans	Public internet targets, real organizations, or third-party services
GitHub Actions workflow, SAST, SCA, secret scanning, Trivy	Unauthorized testing, exploitation, or data access
Kind Kubernetes manifests, RBAC, NetworkPolicy, Pod Security	Production clusters, real cloud accounts, or customer data
ArgoCD, Cosign, Syft, Gatekeeper, Falco, Prometheus, Grafana	Credential harvesting or persistence outside the lab

Risk ratings use a CVSS-style model adapted for a private lab. Severity considers attack complexity, privileges required, reachability, confidentiality impact, integrity impact, availability impact, blast radius, and whether the issue could chain into broader compromise in a realistic enterprise environment.

Severity	Definition Used
Critical	Direct or realistic path to command execution, container escape pattern, cluster compromise, or full administrative control.
High	Significant unauthorized access, data exposure, exploitable dependency risk, or major security-control weakness.
Medium	Governance, monitoring, drift, or metadata control gap that increases operational or audit risk.
Low	Hardening recommendation with limited direct exploitability.

3. Findings Summary

ID	Finding	Severity	Score	Status
SF-001	SQL Injection in Flask /user Endpoint	High	8.1	Documented
SF-002	Command Injection in Flask /ping Endpoint	Critical	9.0	Documented
SF-003	Vulnerable Python Dependency Exposure	High	7.5	Mitigated by Pipeline Control
SF-004	Container Image Hardening and Vulnerability Gap	High	7.2	Remediated
SF-005	Insecure Kubernetes Security Context	Critical	9.1	Remediated
SF-006	Missing Default-Deny Network Segmentation	High	7.4	Remediated
SF-007	Kubernetes Configuration Drift Without GitOps Control	Medium	6.0	Remediated
SF-008	Unsigned Image and Missing SBOM/Provenance Evidence	High	7.2	Remediated
SF-009	Missing Admission Control for Supply-Chain Metadata	Medium	6.4	Remediated
SF-010	Privileged Pod with HostPath Container Escape Pattern	Critical	9.3	Remediated
SF-011	Overprivileged ServiceAccount with cluster-admin Access	Critical	9.0	Remediated
SF-012	Runtime Security and Observability Visibility Gap	Medium	6.2	Remediated

Risk Pattern	The findings show an end-to-end compromise chain: vulnerable code can lead to container compromise; weak container and Kubernetes controls can increase blast radius; poor RBAC can create cluster compromise; missing supply-chain and monitoring controls reduce trust and detection capability.
---------------------	--

Code Risk	Dependency Risk	Image Risk	Cluster Risk	Supply Chain Risk	Runtime Risk
-----------	-----------------	------------	--------------	-------------------	--------------

4. Consolidated Attack Path

The lab demonstrates how separate weaknesses can chain together. A command injection or SQL injection issue starts at the application layer. If the container is weakly hardened, the impact of application compromise increases. If Kubernetes policies allow privileged pods, hostPath mounts, or overprivileged ServiceAccounts, the blast radius can extend from one workload to the cluster. If GitOps, signing, SBOMs, and monitoring are missing, the organization has weaker governance and detection.

Phase	Attack or Control Failure	Security Consequence
1. Application	SQL injection and command injection	Unauthorized data access or command execution in the app runtime
2. Container	Root or weakly hardened image	Greater post-exploitation impact inside the container
3. Kubernetes	Privileged pod, hostPath, weak securityContext	Container isolation bypass pattern and node filesystem exposure
4. RBAC	ServiceAccount bound to cluster-admin	Cluster-wide secret access and administrative control
5. GitOps	Manual cluster drift without self-healing	Unapproved changes persist outside Git review
6. Supply Chain	Unsigned image and missing SBOM/provenance	Low artifact trust and weak vulnerability traceability
7. Detection	No runtime dashboard or alerting	Suspicious behavior may remain unseen

5. Detailed Findings

SF-001	SQL Injection in Flask /user Endpoint		High	8.1	Documented
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:L				
Affected Component	Flask application route /user				
Category	Application Security - Injection				
Testing Environment	Private VM lab				
Evidence Reference	035-sql-injection-proof.png				

Finding Description

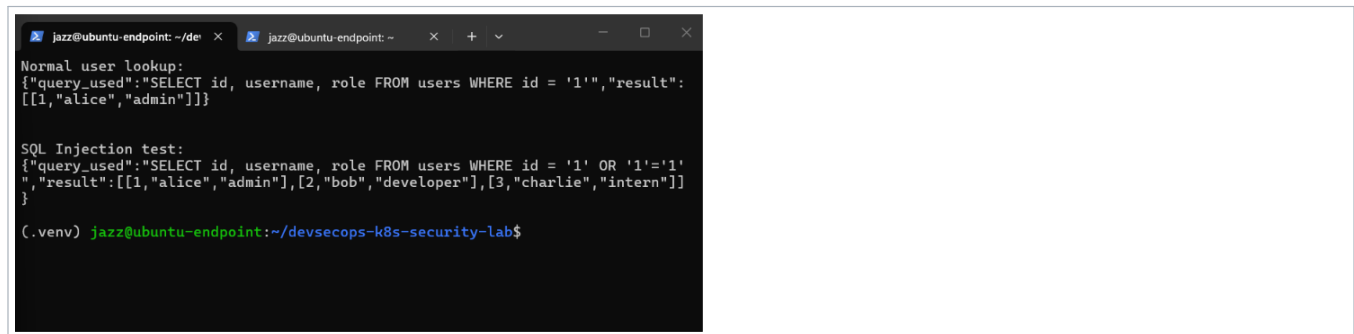
The /user endpoint constructed a SQL query using untrusted user input from the id parameter. This allowed the supplied value to modify SQL logic and return records beyond the intended single-user lookup.

Attack Scenario

A payload such as 1' OR '1'='1 can convert a targeted lookup into a broader query condition. In the lab, this demonstrated unauthorized enumeration of user records.

Technical Evidence

```
cursor.execute("SELECT id, username, role FROM users WHERE id = ?", (user_id,))
```



Evidence for SF-001: SQL Injection in Flask /user Endpoint

Potential Impact

- Unauthorized disclosure of application records.
- Bypass of intended access logic.
- Potential expansion to data tampering if write queries were introduced.
- Regulatory and reputational risk in a real customer-facing system.

Recommended Remediation

- Replace string interpolation with parameterized queries.
- Validate id as an integer before database access.
- Add unit tests for injection payloads.
- Keep SAST checks in CI to detect future unsafe SQL patterns.

Retest / Validation Criteria

Retest should show that the same payload is treated as data, not SQL logic. The endpoint should return one authorized record or a validation error.

SF-002	Command Injection in Flask /ping Endpoint		Critical	9.0	Documented
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H				
Affected Component	Flask application route /ping				
Category	Application Security - Command Execution				
Testing Environment	Private VM lab				
Evidence Reference	036-command-injection-proof.png				

Finding Description

The /ping endpoint used user-controlled input inside a shell command with shell=True. This is a command injection pattern because shell metacharacters can be interpreted as additional commands.

Attack Scenario

The lab used a URL-encoded separator and a harmless whoami command to prove command execution inside the application runtime.

Technical Evidence

```
subprocess.run(["ping", "-c", "1", host], shell=False, capture_output=True, text=True)
```



Evidence for SF-002: Command Injection in Flask /ping Endpoint

Potential Impact

- Arbitrary command execution inside the container.
- Potential access to local files and environment variables.
- Possible lateral movement if container permissions are excessive.
- High business risk if the application handles sensitive data or secrets.

Recommended Remediation

- Remove shell=True.
- Pass command arguments as a list.
- Strictly validate host input using an IP address or hostname allowlist.
- Return controlled errors for invalid input.
- Use network diagnostics in a restricted helper service if needed.

Retest / Validation Criteria

Retest should confirm that command separators are rejected and no injected command output is returned.

SF-003	Vulnerable Python Dependency Exposure		High	7.5	Mitigated by Pipeline Control
CVSS v3.1 Vector		AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N			
Affected Component		Python dependencies and requirements.txt			
Category		Software Composition Analysis			
Testing Environment		Private VM lab			
Evidence Reference		042-pip-audit-dependency-findings.png			

Finding Description

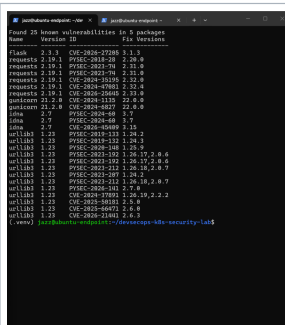
The application intentionally used an outdated dependency to demonstrate software composition analysis. pip-audit identified dependency-level vulnerabilities that could affect confidentiality, integrity, or availability depending on exploit path.

Attack Scenario

An attacker could target known package vulnerabilities if the vulnerable library is reachable through the application code path or indirect dependency behavior.

Technical Evidence

```
pip-audit -r src/app/requirements.txt
```



Evidence for SF-003: Vulnerable Python Dependency Exposure

Potential Impact

- Exposure to known CVEs.
- Increased exploitability through public vulnerability knowledge.
- Pipeline failure or delayed release if dependency hygiene is not controlled.
- Audit weakness when dependency risk is not tracked.

Recommended Remediation

- Upgrade vulnerable dependencies to fixed versions.
- Pin dependencies intentionally and review transitive packages.
- Run pip-audit in CI on every pull request.
- Define severity gates for production releases.
- Generate dependency reports as workflow artifacts.

Retest / Validation Criteria

CI dependency scanning should pass with no unresolved high or critical findings, or findings should be documented with accepted risk and compensating controls.

SF-004	Container Image Hardening and Vulnerability Gap		High	7.2	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L				
Affected Component	Dockerfile and container image				
Category	Container Security				
Testing Environment	Private VM lab				
Evidence Reference	067-docker-image-non-root-user.png				

Finding Description

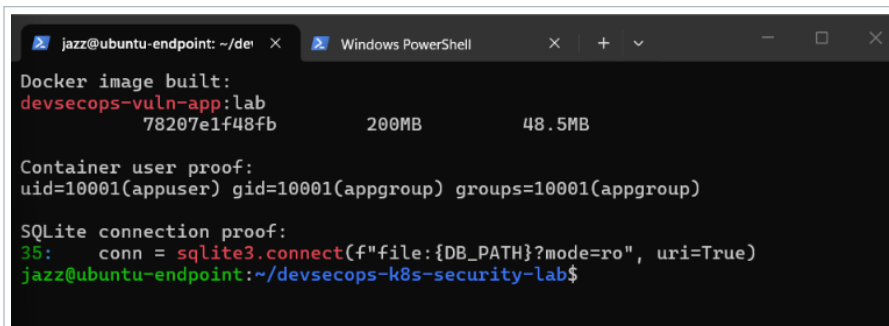
The original container security posture required hardening to reduce runtime risk. Controls were implemented through a multi-stage Dockerfile, non-root UID, Gunicorn runtime, no pip cache, and healthcheck configuration.

Attack Scenario

Weak container images can increase blast radius by running as root, including unnecessary build tools, and making exploitation easier after application compromise.

Technical Evidence

```
docker run --rm --entrypoint id devsecops-vuln-app:lab
```



The screenshot shows a terminal window with the following output:

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ docker run --rm --entrypoint id devsecops-vuln-app:lab
Docker image built:
devsecops-vuln-app:lab
78207e1f48fb          200MB          48.5MB

Container user proof:
uid=10001(appuser) gid=10001(appgroup) groups=10001(appgroup)

SQLite connection proof:
35: conn = sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True)
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence for SF-004: Container Image Hardening and Vulnerability Gap

Potential Impact

- Container compromise has greater impact if running as root.
- Larger images increase vulnerability surface.
- Missing healthchecks reduce operational reliability.
- Poor image hygiene weakens production readiness.

Recommended Remediation

- Use a multi-stage Dockerfile.
- Run the application as UID/GID 10001.
- Avoid root runtime where possible.
- Install only required packages.
- Scan image with Trivy.
- Use Docker and Kubernetes health checks.

Retest / Validation Criteria

The built image was verified to run as UID 10001 and was scanned with Trivy. Kubernetes manifests used imagePullPolicy and security context controls for the lab.

SF-005	Insecure Kubernetes Security Context		Critical	9.1	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H				
Affected Component	Kubernetes Deployment manifest				
Category	Kubernetes Workload Hardening				
Testing Environment	Private VM lab				
Evidence Reference	086-pod-securitycontext-proof.png				

Finding Description

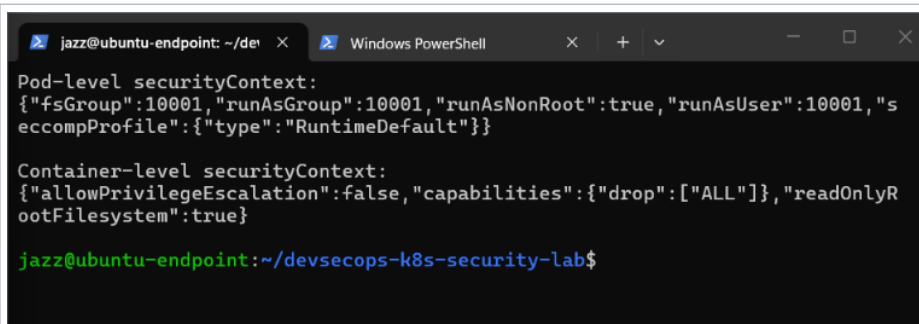
The Kubernetes workload required explicit hardening to prevent root execution, privilege escalation, writeable root filesystem abuse, and unnecessary Linux capabilities.

Attack Scenario

If a workload runs with weak securityContext settings, application compromise may lead to easier privilege escalation inside the container or abuse of kernel features.

Technical Evidence

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  seccompProfile:
    type: RuntimeDefault
```



```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
Pod-level securityContext:
{"fsGroup":10001,"runAsGroup":10001,"runAsNonRoot":true,"runAsUser":10001,"seccompProfile":{"type":"RuntimeDefault"}}

Container-level securityContext:
{"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"]},"readOnlyRootFilesystem":true}

jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence for SF-005: Insecure Kubernetes Security Context

Potential Impact

- Privilege escalation inside container.
- Writeable filesystem persistence attempts.
- Abuse of Linux capabilities.
- Higher likelihood of successful container breakout attempts when chained with other misconfigurations.

Recommended Remediation

- Set runAsNonRoot true.
- Use fixed runAsUser and runAsGroup.
- Disable privilege escalation.
- Drop all Linux capabilities.
- Enable RuntimeDefault seccomp.
- Use readOnlyRootFilesystem true.
- Set CPU and memory requests/limits.

Retest / Validation Criteria

kubectl jsonpath and describe output confirmed that pod and container securityContext controls were applied. A write test to /app failed due to read-only filesystem enforcement.

SF-006	Missing Default-Deny Network Segmentation		High	7.4	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N				
Affected Component	Kubernetes NetworkPolicy				
Category	Kubernetes Network Security				
Testing Environment	Private VM lab				
Evidence Reference	084-networkpolicy-blocked-untrusted-pod.png				

Finding Description

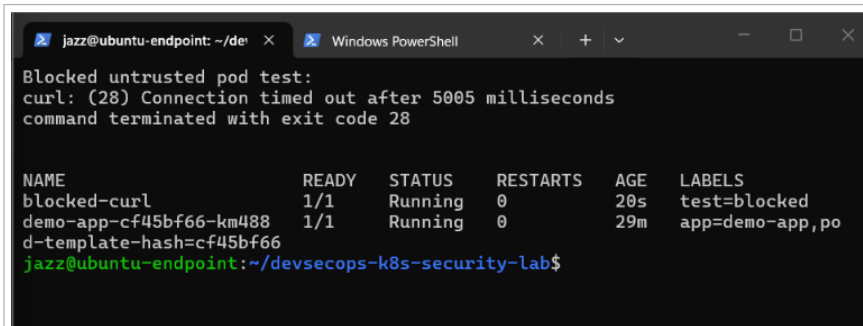
Before NetworkPolicy enforcement, pods could communicate more broadly than required. A zero-trust model was implemented with default-deny and explicit allow policies.

Attack Scenario

An attacker who compromises one pod can attempt lateral movement or service discovery if east-west traffic is unrestricted.

Technical Evidence

```
kubectl get networkpolicy -n devsecops
```



Evidence for SF-006: Missing Default-Deny Network Segmentation

Potential Impact

- Lateral movement between pods.
- Unapproved access to internal services.
- Harder containment during incidents.
- Reduced segmentation between application components.

Recommended Remediation

- Use Calico as NetworkPolicy enforcing CNI.
- Create default-deny ingress and egress policies.
- Allow only approved ingress sources.
- Allow DNS egress explicitly.
- Test with blocked and allowed curl pods.

Retest / Validation Criteria

The untrusted pod was blocked, while trusted/approved pods could access the service. This validated actual segmentation rather than only YAML existence.

SF-007	Kubernetes Configuration Drift Without GitOps Control		Medium	6.0	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:L				
Affected Component	ArgoCD GitOps deployment				
Category	GitOps Governance				
Testing Environment	Private VM lab				
Evidence Reference	103-argocd-self-healed-back-to-git-state.png				

Finding Description

Manual kubectl changes can create drift between the live cluster and the intended Git-defined state. ArgoCD was used to control drift through automated sync and self-healing.

Attack Scenario

An operator or attacker could scale, patch, or modify live objects without Git history if GitOps drift detection is not active.

Technical Evidence

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

```
Waiting for ArgoCD self-heal back to Git-defined replicas...
Check 1/68
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
demo-app  1/1     1            1           19h

NAME      READY   STATUS    RESTARTS   AGE
demo-app-69f8fd4fd-st7p7  1/1     Running   0           32m
demo-app-cf45bf66-km488  0/1     Terminating   0           19h

NAME      SYNC STATUS   HEALTH STATUS
demo-app  Synced       Healthy

Final self-healed state:
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
demo-app  1/1     1            1           19h

Git desired state:
# replicas: 1

ArgoCD status:
NAME      SYNC STATUS   HEALTH STATUS
demo-app  Synced       Healthy
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence for SF-007: Kubernetes Configuration Drift Without GitOps Control

Potential Impact

- Unapproved production configuration changes.
- Loss of auditability.
- Inconsistent environments.
- Security controls bypassed by manual changes.

Recommended Remediation

- Install ArgoCD.
- Create an ArgoCD Application for k8s/base.
- Enable automated sync.
- Enable selfHeal and prune.
- Demonstrate drift correction with manual replica changes.

Retest / Validation Criteria

A manual scale to 3 replicas was reverted by ArgoCD back to the Git-defined desired state.

SF-008	Unsigned Image and Missing SBOM/Provenance Evidence		High	7.2	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N				
Affected Component	GHCR image, Cosign, Syft, provenance				
Category	Supply Chain Security				
Testing Environment	Private VM lab				
Evidence Reference	115-cosign-signature-verified.png				

Finding Description

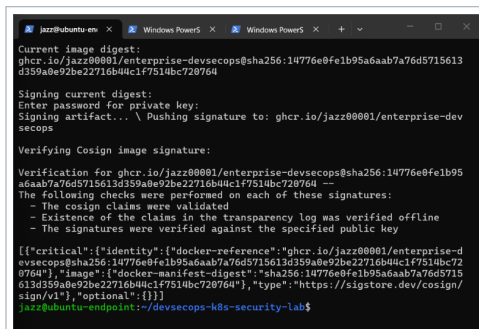
Before signing and SBOM generation, the container image lacked cryptographic evidence of origin, integrity, and component inventory. Cosign and Syft were used to generate supply-chain evidence.

Attack Scenario

Without signing and SBOMs, it is harder to detect tampered images, track vulnerable packages, or prove what artifact was approved for deployment.

Technical Evidence

```
cosign verify --key security/cosign/cosign.pub ghcr.io/<user>/<image>:latest
```



Evidence for SF-008: Unsigned Image and Missing SBOM/Provenance Evidence

Potential Impact

- Untrusted or tampered artifacts may be deployed.
- Weak vulnerability response due to missing component inventory.
- Audit failure for software supply-chain evidence.
- Reduced confidence in CI/CD provenance.

Recommended Remediation

- Push image to GHCR.
- Sign the image with Cosign.
- Verify the signature with the public key.
- Generate SPDX and CycloneDX SBOMs using Syft.
- Attach and attest SBOM.
- Generate SLSA-style provenance for lab evidence.

Retest / Validation Criteria

Cosign signature verification, SBOM generation, SBOM attestation verification, and provenance attestation verification were completed and documented.

SF-009	Missing Admission Control for Supply-Chain Metadata		Medium	6.4	Remediated
CVSS v3.1 Vector		AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N			
Affected Component		OPA Gatekeeper policies			
Category		Policy-as-Code			
Testing Environment		Private VM lab			
Evidence Reference		124-gatekeeper-constraint-created.png			

Finding Description

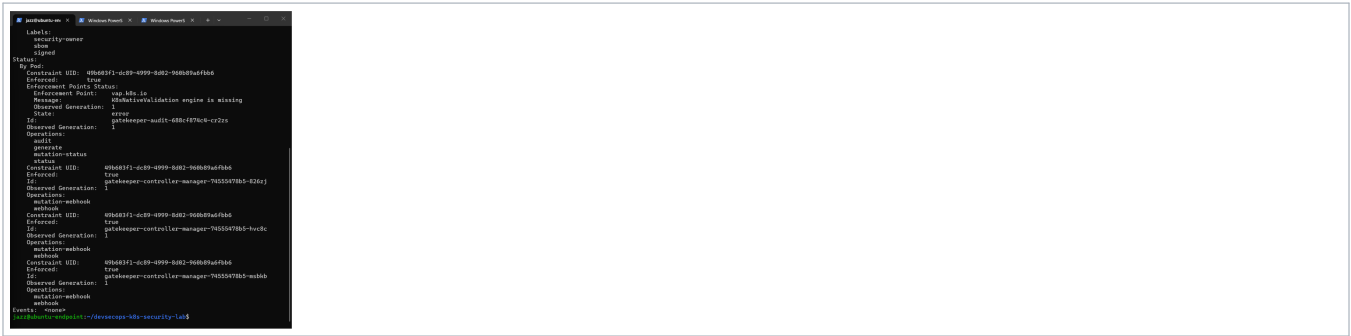
Kubernetes deployments could be created without supply-chain metadata labels. OPA Gatekeeper was implemented to enforce required metadata at admission time.

Attack Scenario

Without metadata controls, teams may deploy workloads without clear ownership, SBOM status, or signing status, making incident response and compliance difficult.

Technical Evidence

```
labels:  
  security-owner: jagriti  
  sbom: syft  
  signed: cosign
```



Evidence for SF-009: Missing Admission Control for Supply-Chain Metadata

Potential Impact

- Poor workload ownership tracking.
- Weak auditability of supply-chain controls.
- Inconsistent deployment governance.
- Reduced incident response speed.

Recommended Remediation

- Install OPA Gatekeeper.
- Create a required-labels ConstraintTemplate.
- Create a Constraint requiring security-owner, sbom, and signed labels.
- Validate non-compliant and compliant deployments.

Retest / Validation Criteria

Gatekeeper policy objects were created and a compliant deployment was allowed. Non-compliant deployment attempts were blocked during policy testing.

SF-010	Privileged Pod with HostPath Container Escape Pattern		Critical	9.3	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H				
Affected Component	Privileged pod and hostPath mount				
Category	Runtime / Kubernetes Escape Pattern				
Testing Environment	Private VM lab				
Evidence Reference	135-container-escape-chroot-proof.png				

Finding Description

A privileged pod with a hostPath mount was used to simulate a container escape pattern. The pod could view the node filesystem through /host and chroot into the mounted node root.

Attack Scenario

A pod using privileged: true and hostPath path / can break isolation assumptions and interact with host/node filesystem content.

Technical Evidence

```
kubectl label namespace runtime-lab pod-security.kubernetes.io/enforce=restricted --overwrite
```

```

Container escape simulation using privileged pod + hostPath
Inside attacker container:
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
Can see mounted node filesystem:
LICENSES
bin
boot
dev
etc
home
kind
lib
lib64
media
Node hostname file:
devsecops-lab-control-plane
Chroot into mounted node filesystem:
root
Now running inside mounted node root:
root
attacker-privileged /
jag@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence for SF-010: Privileged Pod with HostPath Container Escape Pattern

Potential Impact

- Host/node filesystem access from container.
- Potential credential or configuration disclosure.
- Container isolation bypass pattern.
- Potential path to cluster or node compromise when chained with other weaknesses.

Recommended Remediation

- Delete privileged pods.
- Enforce Pod Security restricted mode.
- Prohibit privileged containers and hostPath mounts.
- Use Falco to detect suspicious file access.
- Use admission policies to prevent recurrence.

Retest / Validation Criteria

The privileged pod was detected through Falco during simulation and later blocked after Pod Security restricted labels were applied.

SF-011	Overprivileged ServiceAccount with cluster-admin Access		Critical	9.0	Remediated
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H				
Affected Component	Kubernetes RBAC				
Category	RBAC Privilege Escalation				
Testing Environment	Private VM lab				
Evidence Reference	140-rbac-overprivileged-sa-can-get-secrets.png				

Finding Description

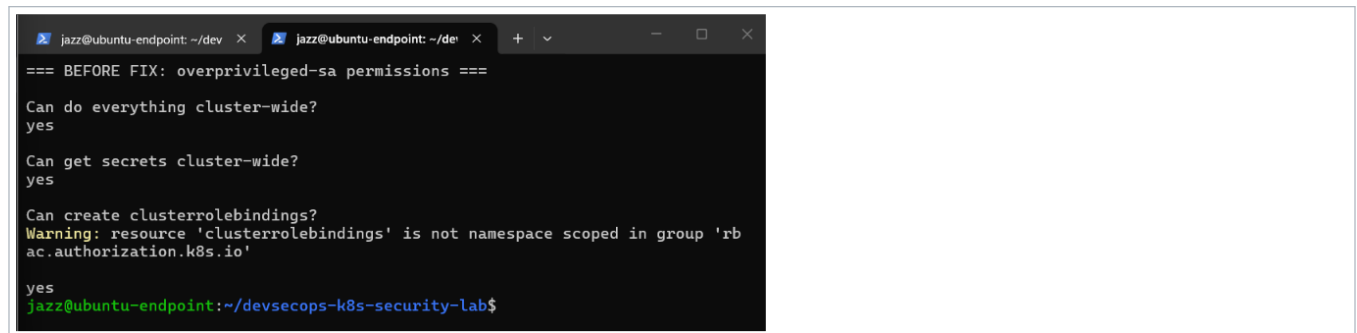
An intentionally overprivileged ServiceAccount was bound to cluster-admin through a ClusterRoleBinding. This demonstrated how a compromised ServiceAccount can become a full cluster compromise path.

Attack Scenario

An attacker controlling this ServiceAccount could read secrets, create powerful bindings, modify workloads, or disable controls across the cluster.

Technical Evidence

```
kubectrl auth can-i get secrets --as=system:serviceaccount:rbac-lab:overprivileged-sa --all-namespaces
```



```
jazz@ubuntu-endpoint: ~/dev x jazz@ubuntu-endpoint: ~/de x + v - □ x
=== BEFORE FIX: overprivileged-sa permissions ===
Can do everything cluster-wide?
yes
Can get secrets cluster-wide?
yes
Can create clusterrolebindings?
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rb
ac.authorization.k8s.io'
yes
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence for SF-011: Overprivileged ServiceAccount with cluster-admin Access

Potential Impact

- Cluster-wide secret access.
- Ability to create additional privileged roles.
- Full cluster administration risk.
- Potential compromise of all namespaces and workloads.

Recommended Remediation

- Delete the cluster-admin ClusterRoleBinding.
- Create namespace-scoped Roles.
- Bind only the required permissions through RoleBinding.
- Audit cluster-admin bindings regularly.
- Verify with `kubectrl auth can-i`.

Retest / Validation Criteria

After remediation, the ServiceAccount could list ConfigMaps in its namespace but could not read secrets or create ClusterRoleBindings.

SF-012	Runtime Security and Observability Visibility Gap		Medium	6.2	Remediated
CVSS v3.1 Vector		AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:L			
Affected Component		Falco, Falcosecurity, Prometheus, Grafana			
Category		Detection and Monitoring			
Testing Environment		Private VM lab			
Evidence Reference		158-custom-falco-security-dashboard-imported.png			

Finding Description

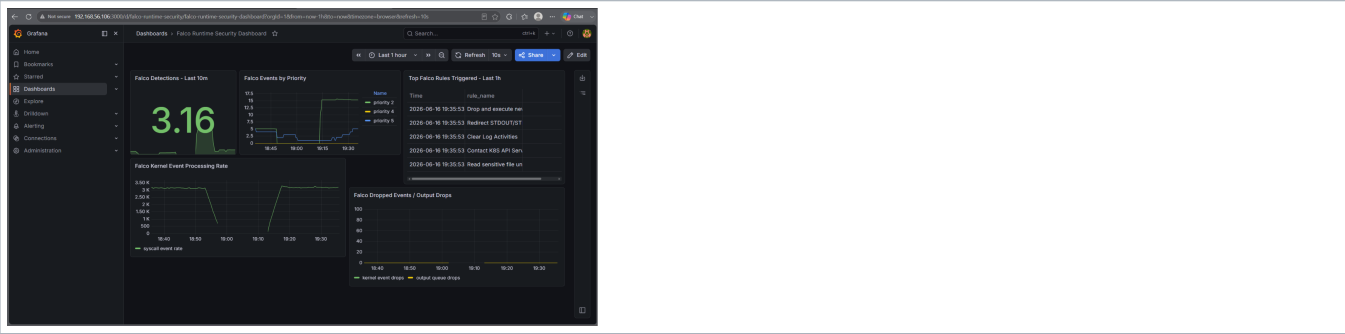
Preventive controls alone do not provide complete security. Falco, Falcosecurity, Prometheus, and Grafana were integrated to create runtime detection and monitoring visibility.

Attack Scenario

Without runtime detection, suspicious actions such as sensitive file access, shell activity, or privileged behavior may not be noticed quickly.

Technical Evidence

```
sum(increase(falcosecurity_falco_rules_matches_total[10m]))
```



Evidence for SF-012: Runtime Security and Observability Visibility Gap

Potential Impact

- Reduced incident detection capability.
- Missed runtime indicators of compromise.
- Weak operational visibility.
- Delayed response to suspicious container activity.

Recommended Remediation

- Install Falco for runtime detection.
- Enable Falcosecurity and metrics.
- Deploy Prometheus and Grafana.
- Create ServiceMonitors.
- Build a custom Falco dashboard.
- Create Prometheus alert rules for detections and dropped events.

Retest / Validation Criteria

Prometheus detected Falco metrics and Grafana displayed runtime security dashboard panels. Falco events were generated and observed in the monitoring flow.

6. Consolidated Remediation Roadmap

The following roadmap converts individual findings into a practical remediation plan. This format is useful for showing engineering prioritization and security program maturity.

Priority	Remediation Action
Immediate	Remove command injection and SQL injection patterns; parameterize database queries; remove shell=True; validate inputs.
Immediate	Delete cluster-admin binding from application ServiceAccounts and replace with namespace-scoped least-privilege RoleBindings.
Immediate	Enforce Pod Security restricted mode and block privileged pods, hostPath mounts, root execution, and privilege escalation.
Short Term	Run Bandit, pip-audit, Trivy image scanning, Trivy config scanning, and secret scanning in CI/CD.
Short Term	Harden images and workloads with non-root UID, read-only root filesystem, dropped capabilities, seccomp RuntimeDefault, probes, and resource limits.
Short Term	Deploy Calico NetworkPolicies using default-deny ingress/egress and explicit allow rules.
Medium Term	Operate Kubernetes deployments through ArgoCD with automated sync, pruning, and self-healing.
Medium Term	Sign images with Cosign, generate Syft SBOMs, and verify SBOM/provenance attestations before release.
Medium Term	Enforce supply-chain metadata and ownership labels through OPA Gatekeeper admission control.
Ongoing	Monitor Falco detections in Prometheus/Grafana, audit RBAC periodically, and rescan dependencies and images on every release.

7. Retest and Validation Matrix

Finding	Validation Criteria
SF-001	Injection payload no longer alters SQL query logic; parameterized query verified.
SF-002	Shell metacharacters rejected; injected commands do not execute; shell=True removed.
SF-003	pip-audit shows no unresolved high/critical dependency findings, or accepted risk is documented.
SF-004	Image runs as non-root UID 10001; Trivy image scan reviewed; Dockerfile uses multi-stage design.
SF-005	Pod and container securityContext show runAsNonRoot, readOnlyRootFilesystem, dropped capabilities, and seccomp RuntimeDefault.
SF-006	Untrusted pod is blocked by NetworkPolicy; trusted pod can reach only the intended service.
SF-007	ArgoCD detects manual drift and restores Git-defined desired state.
SF-008	Cosign verify and verify-attestation succeed; SBOM exists in SPDX and CycloneDX formats.
SF-009	Gatekeeper policy exists; compliant deployment passes; non-compliant deployment is denied.
SF-010	Privileged hostPath pod is blocked by Pod Security; Falco alert evidence captured for simulation.
SF-011	kubectl auth can-i confirms ServiceAccount cannot read secrets or create ClusterRoleBindings.
SF-012	Prometheus has Falco targets up; Grafana dashboard displays runtime security metrics.

8. Control Mapping

This mapping shows how each remediation improves a security-control domain. It is not a formal compliance audit, but it translates the lab into enterprise security language.

Control Domain	Implemented Evidence
Secure Coding	Parameterized queries, shell execution removal, input validation, SAST checks
Dependency Governance	pip-audit SCA, version pinning, workflow artifacts, remediation tracking
Container Security	Multi-stage image, non-root execution, healthcheck, Trivy image scanning
Kubernetes Hardening	SecurityContext, Pod Security restricted, NetworkPolicy, RBAC least privilege, resource limits
GitOps Governance	ArgoCD Application, self-healing, pruning, drift detection, Git-defined desired state
Supply Chain Security	GHCR publishing, Cosign signatures, Syft SBOMs, signed attestations, provenance
Policy-as-Code	OPA Gatekeeper ConstraintTemplate and Constraint for supply-chain labels
Runtime Detection	Falco detections, Falcosidekick metrics, Prometheus queries, Grafana dashboards

9. Final Security Posture

After remediation, the project demonstrates a layered DevSecOps security model: code is scanned, dependencies are audited, images are hardened and signed, Kubernetes objects are deployed with restrictive controls, changes are reconciled through GitOps, admission policies enforce metadata, and runtime behavior is monitored through Falco, Prometheus, and Grafana.

Assessment Conclusion	The lab successfully demonstrates the full professional cycle expected in security engineering: finding discovery, exploit validation, risk assessment, remediation implementation, retest proof, and final reporting.
------------------------------	--

This Security Findings Report is authored for the Enterprise DevSecOps Security Lab by Jagriti Banerjee and is intended for professional portfolio and interview use.

Appendix A - Evidence Reference Index

Evidence File	Purpose
035-sql-injection-proof.png	SQL injection validation for SF-001
036-command-injection-proof.png	Command injection validation for SF-002
042-pip-audit-dependency-findings.png	Dependency vulnerability scan evidence for SF-003
067-docker-image-non-root-user.png	Non-root image validation for SF-004
086-pod-securitycontext-proof.png	Kubernetes SecurityContext validation for SF-005
084-networkpolicy-blocked-untrusted-pod.png	Blocked untrusted pod validation for SF-006
085-networkpolicy-allowed-trusted-pod.png	Allowed trusted pod validation for SF-006
103-argocd-self-healed-back-to-git-state.png	ArgoCD self-healing validation for SF-007
115-cosign-signature-verified.png	Cosign signature verification for SF-008
117-sbom-generated-with-syft.png	Syft SBOM generation for SF-008
119-sbom-attestation-verified.png	SBOM attestation verification for SF-008
124-gatekeeper-constraint-created.png	OPA Gatekeeper Constraint validation for SF-009
127-gatekeeper-allowed-good-deployment.png	Compliant deployment allowed by Gatekeeper for SF-009
135-container-escape-chroot-proof.png	Privileged pod host access simulation for SF-010
136-falco-detected-privileged-pod-activity.png	Falco alert validation for SF-010
138-privileged-pod-blocked-by-pod-security.png	Pod Security restricted enforcement for SF-010
140-rbac-overprivileged-sa-can-get-secrets.png	RBAC cluster-admin abuse validation for SF-011
143-rbac-fixed-cannot-get-secrets.png	Least-privilege RBAC validation for SF-011
154-prometheus-falco-targets-up.png	Prometheus scraping Falco metrics for SF-012
158-custom-falco-security-dashboard-imported.png	Grafana runtime security dashboard for SF-012

Appendix B - Tools Used

Tool	Role in Assessment
Docker	Container build, local runtime testing, image tagging and pushing
Flask	Deliberately vulnerable demonstration application
Bandit	Static application security testing for Python code
pip-audit	Python dependency vulnerability scanning
Trivy	Container image, filesystem, and Kubernetes manifest scanning
Kind	Private Kubernetes cluster inside Docker
kubectrl	Kubernetes validation, testing, and evidence collection
Helm	Installation of ArgoCD, Gatekeeper, Falco, Prometheus, and Grafana components
ArgoCD	GitOps deployment, automated sync, pruning, and self-healing
Cosign	Container image signing and attestation verification
Syft	Software Bill of Materials generation
OPA Gatekeeper	Kubernetes admission control and policy-as-code
Falco	Runtime security detection
Prometheus	Security metrics collection
Grafana	Security dashboard visualization